

并行计算期末复习

文章目录

推动并行计算的因素	
并行计算的应用	
超算机	
并行计算软件技术面临的挑战	
并行程序设计的复杂性	
数据移动(通信)代价很高	
能耗挑战	
伸缩性挑战	
软件生态环境几乎停滞	
Cache相关工作原理及概念	
冯诺伊曼结构	
Cache 高速缓冲存储器(缓存)	
虚拟内存	
低层次的并行 (硬件级别, 程序员不可控)	
并行 多线程相关概念	
硬件级别的单核多线程	
Flynn分类法及相关概念	
SIMD	
SIMD的应用--向量处理器 和 GPU	
MIMD(共享内存, 分布式内存, 网络连接)	
共享内存系统	
一致内存访问(UMA)	
非一致内存访问(NUMA)	
分布式内存系统	
互连网络	
共享内存的互连网络	
分布式内存的互连网络	
直接互联	
等分宽度	
带宽, 等分带宽	
全相连的互连网络	
超立方体互连网络 (Hypercubes)	
间接互联	
延迟和带宽	
共享内存系统缓存一致性	
并行算法设计(竞争条件, 数据依赖, 同步)	
设计要求	
如何设计	
概念定义	
额外开销	
并行算法性能分析(加速比, 效率, 可扩展性, 阿姆达尔定律)	
性能评价标准	
阿姆达尔定律	
效率	
可扩展性	
SIMD编程	
SIMD编程的额外开销	

打包/ 解包数据的开销	
对齐开销	
控制流导致额外开销	
SIMD编程	
Pthread编程	
基础API	
同步相关概念	
忙等待	
互斥量	
信号量同步	
路障(Barrier)	
条件变量	
读写锁	
负载均衡 / 任务分配	
总结	
OpenMP	
编译指示	
积分法求和	
OpenMP 归约	
一些定义, 定理	
依赖关系基本定理	
pragma omp for 指针针对没有循环依赖的内部两个循环	
循环调度	
MPI编程	
基本原语	
通信	
阻塞通信模式	
两种编程模型	
组通信	
广播和归约	
$n-1$ $1-n$	
$n-n$	
Scatter 和 Gather	
非阻塞通信	
混合编程	

推动并行计算的因素

- cpu**晶体管集成密度一直在指数级增长**(符合摩尔定律), 但是**单个cpu的时钟频率2003之后急剧放缓**, 说明单核的时钟频率已不是处理器发展的主角
- 功耗/ 散热 的限制使得单个cpu的时钟频率无法在提升, 只能选择增加cpu数量
- 多核, 众核称为之后cpu的发展趋势, 转变之前“更复杂的处理器设计, 更快的时钟频率”

硬件因素, 单核cpu的发展使得性能的提升受到限制, 导致不得不向多核, 众核发展.

并行计算的应用

- 航空领域, 数据库, 能源, 环境, 游戏, 地理, 药物, 科研, 天气, 生物
- 科学仿真
 - 全局气候建模
 - 海洋建模
 - 星系演化
 - 生物信息学
 - 天文学

- 强子对撞机
- 商用领域：运算能力更强, 速度更快, 价格更便宜

超计算机

- 神威·太湖之光 2016年, 无锡 12.5**亿亿次/秒**
 - 自研的“神威26010”众核处理器
- Summit(顶点) 2018年 IBM研发 20**亿亿次/秒** 超过神威·太湖之光 约60%
- 下一步规划：E级超算 (百亿亿次)

并行计算软件技术面临的挑战

并行程序设计的复杂性

- 足够的并发度
- 并发粒度
 - 独立的计算任务的大小
- 负载均衡
 - 处理器的工作量相近
- 协调和同步
- 局部性
 - 对临近的数据进行计算

数据移动(通信)代价很高

- 因此在设计算法和软件时, 要尽可能地**最小化数据移动**; **每单位数据移动要做更多的计算**.

能耗挑战

- 每MW 需要1M美元

伸缩性挑战

- 相同的程序在**新一代硬件架构**下仍能高效运行.
- 在****更大规模(更多核心)****的硬件平台下仍然能够高效运行(甚至提升)

软件生态环境几乎停滞

- 现有代码还未准备好硬件架构的改变
- 新软件技术还不成熟.

Cache相关工作原理及概念

冯诺伊曼结构

- 五部分: 控制器, 运算器, 存储器, 输入设备, 输出设备
- 控制器和运算器继承在cpu中, 使用不同的寄存器实现对应功能, ALU(算数逻辑单元)和程序计数器
- 缺点: **CPU和存储单元分离设计**, 导致一条指令的执行过程中, cpu从主存中存取数据的过程耗时较大

Cache 高速缓冲存储器(缓存)

- cpu对缓存中的数据进行存取花费时间更短
- 缓存可以集成在cpu上, 也可以在cpu外
- 缓存存储依照的准则
 - **空间局部性, 时间局部性**
 - 程序接下来可能会用到的指令和数据 与 刚刚访问过的指令和数据在物理上是临近存放的
- **多级缓存(一般3级)**
 - 离cpu越近的, 速度越快, 容量越少

- 缓存命中, 缺失
 - 命中: 在多级缓存中找到了相应的数据
 - 缺失: 在多级缓存中未找到.
- 往缓存中存放数据时, 并不是一次放一个, 而是放一个数据块(一个高速缓存行).
 - 第一次访问, 缓存中没有, 需要在主存中找到需要使用的数据, 然后把这个数据附近的一块数据都放入缓存中.
- 缓存和主存中数据不一致(缓存一致性)
 - 写直达(write-through): 一旦发生不一致, 立刻相应更改主存中数据
 - 写回(write-back): 将缓存中不一致的数据标记为脏数据, 待整个高速缓存行里的数据都执行完毕后, 再一次性写入主存中.
- 行主序, 列主序对访问数据效率的影响

虚拟内存

- 在磁盘上开辟一块区域当作内存, 暂时访问不到的数据放在上面

低层次的并行 (硬件级别, 程序员不可控)

- 一个核, 充分利用其中各种的寄存器
- 指令集并行
 - 流水线(pipeline): $7 * 1000 \rightarrow 7 + (1000 - 1)$
 - 多发射(multiple issue): 把任务平均分给不同的处理单元, 让它们同时处理 (有一些数据调度, 任务分配额外开销)
 - 静态: 编译时起分配
 - 动态: 运行时动态分配调整 (超标量)
 - 硬件级别, 程序员无法控制
 - 需要预测哪些指令可以同时执行, 如果判断失误, 会有额外的开销

并行 多线程 相关概念

- 进程: 运行着的计算机程序的实体
 - 包含程序, 内存空间, 资源描述符, 安全信息, 状态信息
- 多任务: 单个处理器同时运行多个任务(宏观上, 微观上分时间片)
- 线程: 一个进程被划分成一些相互独立的任务, 每个运行着的独立任务称作线程
 - fork 派生线程, join合并线程, master线程控制整个程序逻辑, 比如线程的派生和合并

硬件级别的单核多线程

- 细粒度: 分时间片平均轮转执行各个线程
 - 优点: 一个线程遇到阻塞时, 别的线程仍然可以继续执行
 - 缺点: 当一个线程阻塞时, 并不能立刻切换另外一个线程, 而是要等待它所分配的时间片时间结束. 造成了一定的资源浪费
 - 细粒度是相对的, 比之前的指令集还是要粗的
- 粗粒度: 不再分时间片, 只有当某个线程遇到较长时间阻塞时才切换另一个线程.
 - 优点: 线程之间切换少了, 切换浪费的时间少了
 - 缺点: 但是线程切换仍然需要浪费时间, 对于阻塞时间的判断也需要浪费一定时间
- 同步多线程(SMY, 超线程): 模拟多核, 让一个核同时(真正意义上的同时)运行多个线程. 通过分配计算资源, 让不同的计算资源运行不同的线程
 - 一个核上不会同时运行太多线程

Flynn分类法及相关概念

- 计算机体系结构的分类
 - SISM: 单指令流 单数据流 (冯诺伊曼系统)
 - SIMD: 单指令流 多数据流
 - 对多个数据同时执行相同的指令. 一个控制单元对应多个计算单元
 - MISD: 多指令流 单数据流
 - 多个控制单元对应单个计算单元
 - MIMD: 多指令流 多数据流

SIMD

- 将数据分配给多个处理单元, 对多组数据执行相同的指令; **数据并行**
 - n 个数据进行计算, m 个算数逻辑单元, 一个控制单元, 将 n 个数据分配给 m 个算数逻辑单元同时进行运算.
- 所有的计算单元执行相同的指令, 有些计算单元甚至虚啊哟空转
- 这些计算单元必须同步操作, 计算快的需要等待
- 计算单元没有指令存储功能
- 适用于数据量大, 计算简单的并行问题, 并不适合于其余比较复杂的并行问题

SIMD的应用-向量处理器 和 GPU

- 运行速度快
- 易于使用

MIMD(共享内存, 分布式内存, 网络连接)

- 由一些完全独立的处理单元或者核构成, 每一个核或者处理单元**都有自己的控制单元和逻辑计算单元**
- SIMD同步执行, MIMD每一个处理器上有自己的时钟, 可以独立执行, 并不同步

共享内存系统

- 一组自治的处理器通过一个互联网络与内存系统进行连接,
- 每一个处理器可以访问内存中的每一块空间
- 处理器之间通过共享数据进行隐式通信.
- 包括一个或多个多核处理器

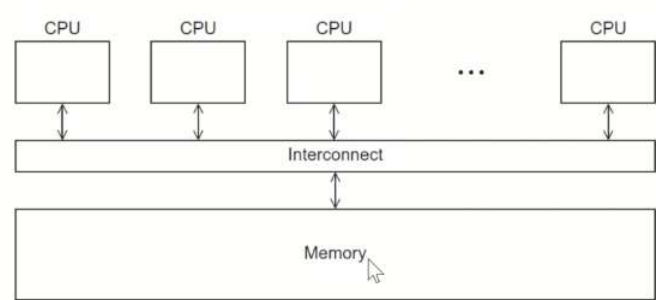


Figure 2.3
https://blog.csdn.net/weixin_40996518

一致内存访问(UMA)

- 一般意义的共享内存系统

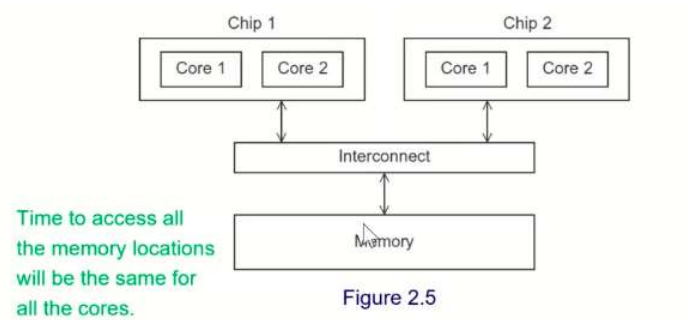
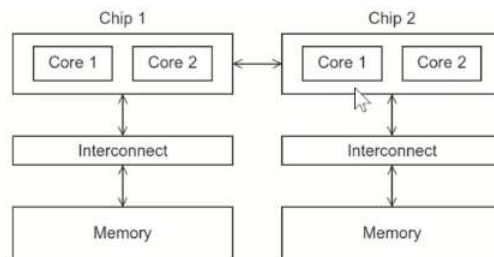


Figure 2.5
https://blog.csdn.net/weixin_40996518

非一致内存访问(NUMA)

- 将每一个多核处理器连接特定的一块内存空间
- 通过处理器中一种特殊的硬件, 互相通信, 从而间接访问所有内存.
- 访问别的内存块时速度较慢



A memory location a core is directly connected to can be accessed faster than a memory location that must be accessed through another chip.

Figure 2.6

分布式内存系统

- 集群
- 每一个CPU单独对应一个内存空间, 构成一节点, 节点之间通过一个互联网络相互连接.

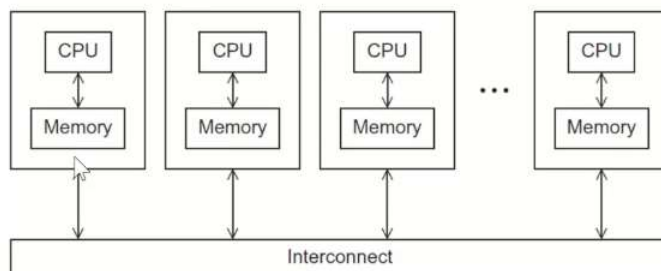


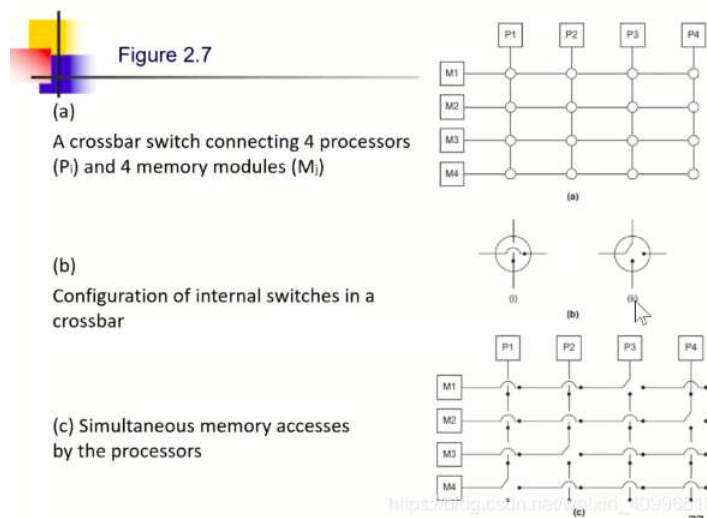
Figure 2.4

https://blog.csdn.net/weixin_40996518

互联网网络

共享内存的互联网络

- 总线
 - 一条总线, 连接各个设备, **同一时刻总线只能被一个设备使用**
 - 当设备数量增多时, 会设计总线争夺, 效率变低
 - 设计简单, 比较灵活, 成本低
- 交叉开关矩阵(**crossbar**)
 - 多个cpu可以同时访问内存
 - 设计复杂, 灵活性差, 成本高



<https://doi.org/10.1016/j.jnlp.2018.04.001>

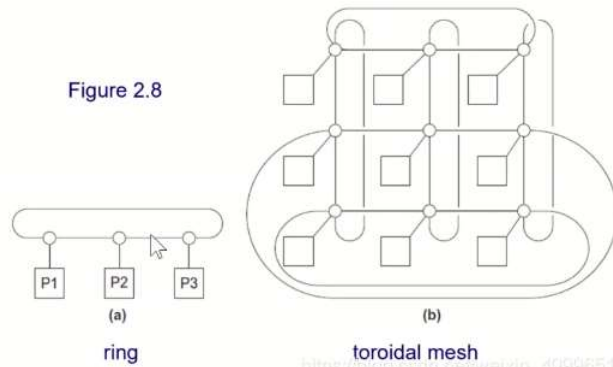
75

分布式内存的互联网络

直接互联

- 1 - 每一个交换器直接连接一个处理器和内存，交换器之间再互相连接
- 2 - ring(环) 和 torodal mesh(二维环形网格)
- 3 - 缺点：两个处理器之间通信时，需要占用别的额外的开关和链路。

Figure 2.8



https://blog.csdn.net/weixin_40996518

等分宽度

- 1 - 用来衡量一个互联网络连通性的标准(同时通信的链路数目)
- 2 - 把节点平分成两组, 看这两组之间同多少个可以同时通信的链路数目, 分组不同, 数目不同, **等分宽度取最小的那个,**
- 3 - **去除最少的链路数目, 使得所有节点均分成两份, 最少的链路数目即为等分宽度**

- 二维网格的等分宽度： $2\sqrt{p}$
- 环状：2

带宽, 等分带宽

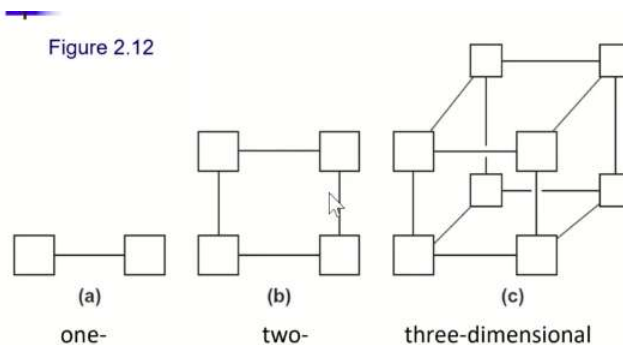
- 带宽：链路上传输数据的速度, MB/s, Mb/s
- 等分带宽：带宽 * 等分宽度, 用来衡量一个网络的数据传输速度

全相连的互联网络

- 等分宽度: $p^2/4$ ($p/2 * p/2$)
- 造价太高, 现实不可能

超立方体互联网络 (Hypercubes)

Figure 2.12



- 等分宽度 : $p/2$
 - 当 $p > 4$, 时超立方体的互连网络优于环和二维环形网格结构

间接互联

- 需要许多交换器用来维护任意两个节点之间的通信
- Crossbar
 - 之前讲的是**共享内存系统**, 处理器和内存块之间的互联结构
 - 现在是**分布式内存系统**, 节点(一个处理器对应一个内存块称为一对)之间的互联方式。
- Omega network

延迟和带宽

- 信息传输时间 = $l + n / b$
- l : 延迟

- n : 数据长度(字节)
- b : 带宽

共享内存系统缓存一致性

- **基于监听的缓存一致性协议** : 当共有变量被修改时, 需要在互联网络上进行一个广播, 别的核将该数据标记为非法, 待重新读取更改后数据时, 才能继续执行后续步骤
- **基于目录的缓存一致性协议** : 用一个目录存放每一个内存行的状态, 每一个高速缓存行的变更都会会在目录中进行记录, 别的核在读取数据之前都会先查看目录.

并行算法设计(竞争条件, 数据依赖, 同步)

- SPMD : 单程序 多数据流.
 - 只包含一份可执行代码, 通过一些分支语句, 表现的像在不同处理器上执行不同的程序.
 - 微观上看不同处理器执行不同的语句, 宏观上看, 执行同一个程序
- **先设计串行算法, 然后将其 改为并行算法**
 - 并不一定是最好的策略——有些情况下, 最优并行算法与最有串行算法之间完全没有关系
 - 但是是一个切实可行的方法

设计要求

- 进程/线程 的**任务分配**
 - **负载均衡** : 每个进程/线程获得大致相同的工作量
 - **通信量尽量少**
- 安排进程/线程之间的**同步**
- 安排进程/线程之间的**通信**

如何设计

- 任务并行 : 输出的内容, 每一个任务需要用到所有数据
- 数据并行 : 输入的数据, 每一个数据需要完成所有任务.
- 混合并行 : 任务 + 数据 并行结合使用
- 数据分解 : 每个任务的计算工作的结果对最终结果都有用, 都要全部昨晚
- 搜索分解 : 一旦一个任务找到解, 全部任务即可停止, 工作量可能大于, 也可能小于串行算法.

概念定义

- **原子性** : 一组操作要么全部执行, 要么全部不执行
- **互斥** : 任何时刻都只有一个线程在执行
- **竞争条件** : 执行结果依赖于两个或更多事件的时序
 - 多个**进程/线程**尝试更新同一个共享资源时, 结果可能是无法预测的.
- **数据依赖** : 两个**内存操作的序**, 为了保证结果的正确性, 必须保持这个序
- **同步** : 在时间上, 强制使各个进程/线程在某一点相互等待, 确保各进程/线程的正常顺序和对共享可写数据的正确访问.
- **障碍** : 阻塞线程继续执行, 在此程序点等待, 直到所有参与线程都到达障碍点才继续执行

额外开销

- 分配计算任务的额外带啊嘛
- 锁开销 : 加锁/ 解锁操作本身开销和线程间竞争导致的空闲等待
- 负载不均

并行算法性能分析(加速比, 效率, 可扩展性, 阿姆达尔定律)

- 串行算法的评价 : 算法时间复杂度标示为输入规模的函数
- 并行算法的评价 : 除了输入规模, 还要考虑**处理器数目**, 处理器相对运算速度, 通信速度.
- 评价标准
 - 运行时间
 - 加速比 : 与最优串行算法做比较

性能评价标准

- 运行时间
 - 串行时间: T_s , 算法开始到结束的时间
 - 并行时间: T_p , 并行算法开始到最后一个进程结束所经历的时间
- 并行算法总额外开销
 - $T_0 = pT_p - T_s$
- 加速比:
 - $S = T_s/T_p$
- p 为并行核数
 - 一般 $S \leq p$
 - $S = p$: **线性加速比**
 - $S > p$: **超线性加速比**, 在实践中可能出现

阿姆达尔定律

- 除非一个串行程序的执行**几乎全部都并行化**, 否则不论多少可利用的核, 通过并行化产生的加速比都是会受限的。
- 加速比只和 a , p 有关, 与串行时间无关

- $S = 1 / (1 - a + a/p)$
- a 为串行程序中可被(完美)并行化的比例
- $T_s = 1$,
- $T_p = T_{\text{不可并行}} + T_{\text{可并行}} = 1 - a + a/p$

- 某串行程序运行时间为20s，可并行化比例为90%，若使用p个核将其并行化，加速比是多少？
 - $T_S = 20\text{ s}$
 - $T_P = T_{\text{不可并行}} + T_{\text{可并行}}$

$$= 0.1 \times T_S + 0.9 \times T_S / p$$

$$= 0.1 \times 20\text{s} + 0.9 \times 20\text{s} / p$$

$$= 2\text{s} + 18\text{s} / p$$
 - $S = T_S / T_P = 20 / (2 + 18 / p) = 10 / (1 + 9 / p)$
 - 或用公式: $S = 1 / (1 - a + a / p) = 1 / (0.1 + 0.9 / p)$

效率

- 把核数引入了评价指标

- 效率 (Efficiency)：度量有效计算时间
- $E = S / p = T_S / (p * T_p)$
- 理想情况=1，正常0~1。
 - 因为理想情况 $S = p$

可扩展性

- 强可扩展性：问题规模不变时，**效率不随着线程数的增大而降低**，则称程序时强可扩展的。
 - 只有程序全部并行化，才能达到。
- 弱可扩展的：问题规模以一定速率增大(退而求其次)，**效率不随着线程数的增大而降低**，则称程序是弱可扩展的。
- 可扩展的：程序核数增加，如果在输入规模也以**相应的增长率增加**的情况下，该程序效率不降，则称是**可扩展的**。

- 某程序串行版本运行时间为 $T_s = n$ 秒, 这里 n 也为问题规模, 该程序某并行版本运行时间为 $T_p = n/p + T_o$ 。该并程序是否可扩展? (假设 T_o 为常数, 不随 p 变化而变化)
 - $E = T_s / p * T_p = \frac{n}{p(n/p + T_o)} = \frac{1}{1 + \frac{p}{n} * \frac{T_o}{1}}$
 - 要保持 E 不变, 当 $p' = kp$ 时, 只需问题规模等比例增大即可, 即 $n' = kn$ 。
 - 因此, 该并程序时可扩展的。

https://blog.csdn.net/weixin_40996518

- 可扩展性是高性能并行算法追求的主要目标
 - 度量并行系统性能的方法之一
 - 度量并行体系结构在不同系统规模下的并行处理能力
 - 度量并行算法内在的并行性
 - 利用系统规模和问题规模已知的并行系统性能来预测规模增大后的性能

上述评价指标针对的是MIMD(SPMD属于MIMD)

SIMD编程

SIMD编程的额外开销

打包/ 解包数据的开销

- 打包 — 将数据拷贝到连续的内存区域, 然后读进向量寄存器
- 解包 — 将数据对象从向量寄存器拷贝回内存中

除非数据本来就在连续内存, 否则打包/解包开销难以避免

对齐开销

- 对齐的内存访问
 - 读取的起始地址总是向量长度的倍数(例如16字节, 128b)
 - 因此如果存储的数据不再向量长度的倍数位置, 则会额外读取一段, 在进行合并. 造成额外开销
- 未对齐的内存访问
 - 地址不是16字节的倍数
 - 静态对齐: 对未对齐的毒操作, 做两次相邻的对齐毒操作, 然后进行合并.
 - 有时硬件会帮你做, 但仍然会产生多次内存操作.

静态调整循环

```
float a[64];
for (i=0; i<60; i+=4)
    Va = a[i+2:i+5];

float a[64];
Sa2 = a[2]; Sa3 = a[3];
for (i=4; i<64; i+=4)
    Va = a[i:i+3];
```

https://blog.csdn.net/weixin_40996518

未对齐的内存访问

- 距16字节边界的偏移是变化的或未知的
- 动态对齐：合并点在运行时计算

```
float a[64];
start = read();
for (i=start; i<60; i+=4)
    Va = a[i:i+3];
```

➡

```
float a[64];
start = read();
off = start % 4;
for (i=start; i<60; i+=4)
    V1 = a[i-off:i-off+3];
    V2 = a[i-off+4:i-off+7];
    Va = merge(V1, V2, off*4);
```

控制流导致额外开销

- 如果程序中有控制流变化, 就需要**所有路径都执行**, 因此造成额外开销

```
for (i=0; i<16; i++)
    if (a[i] != 0)
        b[i]++;
```

What happens:

对所有元素计算 $a[i] \neq 0$
 计算所有元素的 $b[i]++$ 存入临时寄存器t1
 将原 $b[i]$ 拷贝到寄存器t2
 根据 $a[i] \neq 0$ 的结果合并t1和t2

```
for (i=0; i<16; i++)
    if (a[i] != 0)
        b[i] = b[i] / a[i];
    else
        b[i]++;
```

What happens:

对所有元素计算 $a[i] \neq 0$
 计算所有 $b[i] = b[i]/a[i]$ 存入t1
 计算所有 $b[i]++$ 存入t2
 根据 $a[i] \neq 0$ 的结果合并t1和t2

https://blog.csdn.net/welxin_40996518

- 假定所有控制流路径执行频率都不同
- 应该针对频率最高的路径优化代码
- 其他路径按默认方式执行

```
for (i=0; i<16; i++)
    if (a[i] != 0)
        b[i]++;
```

若a多数不为0可以怎么做(特例):

对满足向量 $\mathbf{a}[i] \neq 0$ 的分支进行向量化, 否则按常规方式执行

```
for (i=0; i<16; i+=4){
    if(a[i:i+3] != (0, 0, 0, 0))
        b[i:i+3] = b[i:i+3] + (1, 1, 1, 1);
    else
        //执行常规操作;
}
```

但是一般不知道每条路径的执行频率,一般认为控制程序不适合向量化

SIMD编程

- SSE指令集：8个128位向量寄存器

```
void mul(int n, float a[][maxN], float b[][maxN], float c[][maxN]) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            c[i][j] = 0.0;
            for (int k = 0; k < n; ++k) {
                c[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}
```

https://blog.csdn.net/weixin_40998655

- 转秩, 缓存优化


```
void trans_mul(int n, float a[][maxN], float b[][maxN], float
c[][maxN]){
    // transposition
    for (int i = 0; i < n; ++i) for (int j = 0; j < i; ++j) swap(b[i][j], b[j][i]);
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            c[i][j] = 0.0;
            for (int k = 0; k < n; ++k) {
                c[i][j] += a[i][k] * b[j][k];
            }
        }
    }
    // transposition
    for (int i = 0; i < n; ++i) for (int j = 0; j < i; ++j) swap(b[i][j], b[j][i]);
}
```

```
void sse_mul(int n, float a[][maxN], float b[][maxN], float c[][maxN]){
    __m128 t1, t2, sum;
    for (int i = 0; i < n; ++i) for (int j = 0; j < i; ++j) swap(b[i][j], b[j][i]);
    for (int i = 0; i < n; ++i){
        for (int j = 0; j < n; ++j){
            c[i][j] = 0.0;
            sum = _mm_setzero_ps();
            for (int k = n - 4; k >= 0; k -= 4){ // sum every 4 elements
                t1 = _mm_loadu_ps(a[i] + k);
                t2 = _mm_loadu_ps(b[j] + k);
                t1 = _mm_mul_ps(t1, t2);
                sum = _mm_add_ps(sum, t1); 此时得到了什么？
            }
            sum = _mm_hadd_ps(sum, sum);
            sum = _mm_hadd_ps(sum, sum);
            _mm_store_ss(c[i] + j, sum);
        }
    }
}
```

```

        for (int k = (n % 4) - 1; k >= 0; --k){ // handle the last n%4
elements
            c[i][j] += a[i][k] * b[j][k];
        }
    }
}
for (int i = 0; i < n; ++i) for (int j = 0; j < i; ++j) swap(b[i][j], b[j][i]);
}

```

https://blog.csdn.net/weixin_40995518

- 分块 进入Cache的次数 $2n/T$, 不分块是 $n +$

Pthread编程

- 伪共享：读取数据时,是一个缓存行一个缓存行的读取
 - 当多个处理器访问同一个缓存行时,即使访问的是不同的机器字,看起来也会有竞争条件. 会产生不必要的协同开销.
- Pthread是POSIX标准
 - 相对底层
 - 程序员控制线程管理和协调
 - 程序员分解并行任务并管理任务调度
 - 可移植但比较慢

基础API

```
1 pthread_t 是一种Pthread编程中的数据类型
2 pthread_create(pthread_t *, const pthread_attr_t*, void*(*)(void*), void*);
3 - 第一个参数是线程id
4 - 第二个参数是各种属性, NULL表示默认属性值
5 - 第三个参数是线程要运行的函数(参数和返回值类型都是void*)
6 - 第四个参数是传递给要运行的函数的参数
7
8     1. 主线程借助操作系统创建一个新线程
9     2. 线程执行一个特定函数thread_fun
10    3. 所有创建的线程执行相同的函数, 表示线程的计算任务分解
11    4. 对于程序中不同线程执行不同任务的情况, 可以用创建线程时传递的参数区分线程的id, 以及其他线程的独特特性
12
13 int main()
14 {
15     pthread_t threads[16];
```



```
16     int tn;  
17     for(tn=0; tn < 16; tn++)  
18     {  
19         pthread_create(&threads[tn], NULL, ParFun, NULL); //执行ParFun函数  
20     }  
21     for(tn=0; tn < 16; tn++)  
22     {  
23         pthread_join(threads[tn], NULL);  
24     }  
25     return 0;  
26 }
```

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

/* Global variable: accessible to all threads */
int thread_count;

void* Hello(void* rank); /* Thread function */

int main(int argc, char* argv[]) {
    long thread; /* Use long in case of a 64-bit system */
    pthread_t* thread_handles;

    /* Get number of threads from command line */
    thread_count = strtol(argv[1], NULL, 10);

    thread_handles = (pthread_t*) malloc(thread_count*sizeof(pthread_t));

    for (thread = 0; thread < thread_count; thread++)
        pthread_create(&thread_handles[thread], NULL, Hello, (void*) thread);

    printf("Hello from the main thread\n");

    for (thread = 0; thread < thread_count; thread++)
        pthread_join(thread_handles[thread], NULL);

    free(thread_handles);
    return 0;
} /* main */

void* Hello(void* rank) {
    long my_rank = (long) rank; /* Use long in case of 64-bit system */

    printf("Hello from thread %ld of %d\n", my_rank, thread_count);

    return NULL;
} /* Hello */
```

```
1 void pthread_exit(void* value_ptr)
2 通过value_ptr返回结果给调用者
3
4 int pthread_cancel(pthread_t thread);
5 取消线程thread的执行
6
7 主线程使用pthread_cancel(thread);
8 子线程使用pthread_testcancel()接收信号
```

同步相关概念

- **临界区**：一个更新共享资源的**代码段**，一次只能允许一个线程执行该代码段
- **原子性**：一组操作要么全部执行，要么全部不执行
- **竞争条件**：多个进程/线程尝试更新同一共享资源时，结果可能是无法预测的，则存在竞争条件
- **数据依赖**：两个内存操作的序，为了保证结果的正确性，必须保持这个序
- **同步**：在时间上，使所有进程/线程强制在某一点必须相互等待，确保进程/线程的正常顺序和对共享可写数据的正确访问

$$\pi = 4 \left[1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots \right] = 4 \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}$$

```
double factor = 1.0;  
double sum = 0.0;  
for (k = 0; k < n; k++, factor = -factor)  
    sum += factor/(2*k+1);  
pi_approx = 4.0*sum;
```

```
void *pi_t1_read(void *param)
{
    threadParm_t *p = (threadParm_t *) param;
    int r = p->threadId;
    int n = p->n;
    int my_n = n/THREAD_NUM;
    int my_first = my_n*r;
    int my_last = my_first + my_n;

    if (my_first % 2 == 0) factor = 1.0;
    else factor = -1.0;

    for (int i = my_first; i < my_last; i++, factor = -factor) {
        sum += factor/(2*i+1);
    }
    pthread_exit(NULL);
}
```

忙等待

同步——忙等待方法

```
void *pi_busywaiting(void *param)
{
    threadParm_t *p = (threadParm_t *) param;
    int r = p->threadId; int n = p->n; int my_n = n/THREAD_NUM;
    int my_first = my_n*r; int my_last = my_first + my_n;
    double my_sum = 0.0;
    if (my_first % 2 == 0) factor = 1.0;
    else factor = -1.0;
    for (int i = my_first; i < my_last; i++, factor = -factor) {
        my_sum += factor/(2*i+1);
    }

    while (flag != r) Sleep(0);
    sum += my_sum;
    flag++;
    pthread_exit(NULL);
}
```

flag指出的线程编号才允许累加到全局和

避免过多忙等待
线程先各自求局部和

互斥量

- 创建mutex:

```
#include <pthread.h>

pthread_mutex_t amutex = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_init(&amutex, NULL);
```
- 使用mutex:

```
int pthread_mutex_lock(&amutex); // 加锁, 若已上锁则阻塞至被解锁
int pthread_mutex_trylock(&amutex);

// 加锁, 若已上锁不阻塞, 返回非0
int pthread_mutex_unlock(&amutex); // 解锁, 可能令其他线程退出阻塞
```
- 释放mutex:

```
int pthread_mutex_destroy(&amutex);
```



互斥量版本

```
void *pi_mutex(void *parm)
{
    threadParm_t *p = (threadParm_t *) parm;
    int r = p->threadId;  int n = p->n;  int my_n = n/THREAD_NUM;
    int my_first = my_n*r;  int my_last = my_first + my_n;
    double my_sum = 0.0;
    if (my_first % 2 == 0) factor = 1.0;
    else factor = -1.0;
    for (int i = my_first; i < my_last; i++, factor = -factor) {
        my_sum += factor/(2*i+1);
    }

    pthread_mutex_lock(&mutex);
    sum += my_sum;
    pthread_mutex_unlock(&mutex);
    pthread_exit(NULL);
}
```

https://blog.csdn.net/weixin_40995518

40

- 忙等待和互斥量的区别
 - 互斥量是阻塞等待, 没有占用cpu资源. 而忙等待是依然占用cpu资源的.
 - 忙等待必须指定顺序, 依次执行临界区. 互斥量, 则是操作系统自行决定顺序.
- 死锁

- 持有多个mutex可能导致死锁:

thread1	thread2
lock(a)	lock(b)
lock(b)	lock(a)

- 上锁/打开，二元状态

信号量同步

- ### ● 初始化信号量

```
#include <semaphore.h>
int sem_init(sem_t *sem, int pshared, unsigned value);
```

Pshared 非0: 进程间共享; 0: 进程内线程共享

value: 信号量初始值

- 使用信号量:

```
int sem_wait(sem_t *sem); // 信号量值减1, 若已为0则阻塞
int sem_post(sem_t *sem); // +1, 若原来为0则可能唤醒阻塞线程
```

- 释放信号量:

```
int sem_destroy(sem_t *sem);
```

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

#define NUM_THREADS 4

typedef struct{
    int      threadId;
} threadParm_t;

sem_t  sem_parent;
sem_t  sem_children;
void *threadFunc(void *parm)
{
    threadParm_t  *p = (threadParm_t *) parm;
```

```

printf(stdout, "I am the child thread %d.\n", p->threadId);
sem_post(&sem_parent); // 唤醒主线程——我已完成
sem_wait(&sem_children);
printf(stdout, "Thread %d is going to exit.\n", p->threadId);
pthread_exit(NULL);
}

// 等待主线程唤醒我

int main(int argc, char *argv[])
{
    sem_init(&sem_parent, 0, 0); // 初始值都是0
    sem_init(&sem_children, 0, 0);
    pthread_t      thread[NUM_THREADS];
    threadParm_t    threadParm[NUM_THREADS];
    int            i;

    for (i=0; i<NUM_THREADS; i++)
    {
        threadParm[i].threadId = i;
        pthread_create(&thread[i], NULL, threadFunc, (void
*)&threadParm[i]);
    }

    for (i=0; i<NUM_THREADS; i++)
    {
        sem_wait(&sem_parent); // 等待所有子线程都输出
    }

    printf(stdout, "All the child threads has printed.\n");

    for (i=0; i<NUM_THREADS; i++)
    {
        sem_post(&sem_children); // 唤醒子线程继续输出新内容
    }

    for (i=0; i<NUM_THREADS; i++)
    {
        pthread_join(thread[i], NULL);
    }

    sem_destroy(&sem_parent);
    sem_destroy(&sem_children);
    return 0;
}

```

路障(Barrier)

- 线程数3, 表示当有3个线程到达路障时, 就不用再等待

- 初始化barrier的方法如下所示（本例中线程数为3）：

```
pthread_barrier_t b;  
pthread_barrier_init(&b, NULL, 3);
```

第二个参数指出对象属性，NULL表示默认属性为等待barrier，线程应执行

```
pthread_barrier_wait(&b);
```

可通过下面的宏，指定一个初始值来初始化barrier

```
PTHREAD_BARRIER_INITIALIZER(3).
```

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#define NUM_THREADS 4

typedef struct{
    int    threadId;
}threadParm_t;

pthread_barrier_t barrier;

void *threadFunc(void *parm)
{
    threadParm_t    *p = (threadParm_t *) parm;
    printf(stdout, "Thread %d has entered step 1.\n", p->threadId);
    pthread_barrier_wait(&barrier);
}
```

```
        fprintf(stdout, "Thread %d has entered step 2.\n", p->threadId);
        pthread_exit(NULL);
    }

    int main(int argc, char *argv[])
    {
        pthread_barrier_init(&barrier, NULL, NUM_THREADS);
        pthread_t      thread[NUM_THREADS];
        pthreadParm_t   threadParm[NUM_THREADS];
        int            i;

        for (i=0; i<NUM_THREADS; i++)
        {
            threadParm[i].threadId = i;
            pthread_create(&thread[i], NULL, threadFunc, (void
*&threadParm[i]);
        }

        for (i=0; i<NUM_THREADS; i++)
        {
            pthread_join(thread[i], NULL);
        }

        pthread_barrier_destroy(&barrier);
        system("PAUSE");
        return 0;
    }
```

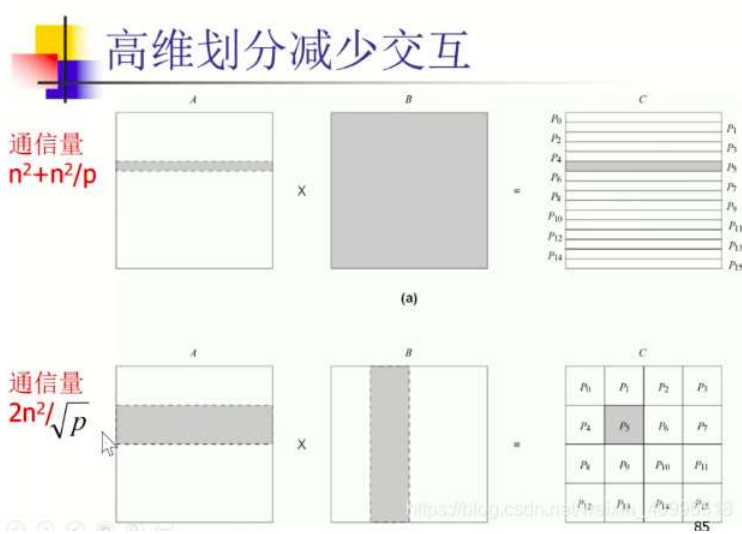
条件变量

读写锁

- 结点级锁
- 对整个链表加锁
 - 额外开销太大
- 读写锁

负载均衡 / 任务分配

- 数据分布不均等且未知, 如果按照大块划分, 会造成划分任务时不均衡.
 - 动态任务分配, 每个线程一开始只分配一部分, 之后哪个线程先完成, 哪个线程继续取任务.
 - 动态划分的块也有讲究
- 块划分
- 二维划分
- 高维划分
- 循环划分
- 随机块划分 – 稀疏矩阵



总结

- 优点

- 支持多种语言
- 之称的语言被程序员所熟悉
- 数据共享方便
- 缺点
 - 数据竞争发现

OpenMP

- 通过少量编译指示指出并行部分和数据共享, 即可实现很多串行程序的并行化
- 可以
 - 程序员只需要将程序分为串行和并行区域, 而不是构建并发执行的多线程
 - 隐藏栈管理
 - 提供同步机制
- 不可以
 - 自动化并行
 - 确保加速
 - 避免数据竞争

编译指示

```
1 | #pragma omp
```

- 共享数据: 所有程序可见, 通常是具名的
- 私有数据: 单线程可见, 通常在栈中分配

```
1 | shared 变量是共享的
2 | private 变量是私有的
3 | 默认是shared
4 | 循环变量是private
5 |
6 | #pragma omp parallel shared(bigdata) private(p)
7 |
8 | 条件编译
9 | #ifdef _OPENMP
10 | #include <omp.h>
11 | #endif
12 |
13 | 任何使用编译指示的地方都需要这两个条件编译
14 |
15 | int omp_get_num_threads(void);
16 | 返回当前并行区域的线程组中的线程数
17 |
18 | int omp_get_thread_num(void);
19 | 返回当前线程在线程组中的编号, 主线程是0, 编号在0-omp_get_num_threads()-1之间
```

```
# ifdef _OPENMP
    int my_rank = omp_get_thread_num ( );
    int thread_count = omp_get_num_threads ( );
# else
    int my_rank = 0;
    int thread_count = 1;
# endif
```

https://blog.csdn.net/weixin_40996518

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Hello(void); /* Thread function */

int main(int argc, char* argv[]) {
    /* Get number of threads from command line */
    int thread_count = strtol(argv[1], NULL, 10);

    #pragma omp parallel num_threads(thread_count)
    Hello();

    return 0;
} /* main */
```

```
void Hello(void) {
    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();

    printf("Hello from thread %d of %d\n", my_rank, thread_count);
} /* Hello */
```

积分法求和

串行版本

```
/* Input: a, b, n */

h = (b - a)/n;
approx = (f(a) + f(b))/2.0;

for (j = 1; j <= n-1; j++)
    approx += f(a + i*h);

approx = h * approx;
```

多线程求全局和怎么保证正确性？

```
1 | # pragma omp critical
```

第一个OpenMP版本

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void Trap(double a, double b, int n, double* global_result_p);

int main(int argc, char* argv[]) {
    double global_result = 0.0;
    double a, b;
    int n;
    int thread_count ;

    thread_count = strtol(argv[1], NULL, 10) ;
    printf("Enter a, b, and n\n");
    scanf("%lf %lf %d", &a, &b, &n);
    # pragma omp parallel num_threads(thread_count)
    Trap(a, b, n, &global_result);
    printf("With n = %d trapezoids, our estimate\n", n);
    printf("of the integral from %f to %f = %.14e\n", a, b, global_result);
    return 0;
} /* main*/
```

25

第一个OpenMP版本(2)

```
void Trap( double a, double b, int n, double* global_result_p ) {
    double h, x, my_result ;
    double local_a , local_b ;
    int i, local_n ;

    int my_rank = omp_get_thread_num();
    int thread_count = omp_get_num_threads();
    h = (b-a)/n;
    local_n = n/thread_count ;
    local_a = a + my_rank*local_n*h;
    local_b = local_a + local_n*h;
    my_result = (f(local_a) + f(local_b))/2.0;
    for (i = 1; i <= local_n - 1; i++) {
        x = local_a + i*h;
        my_result += f(x);
    }
    my_result = my_result*h;
    #pragma omp critical
    *global_result_p += my_result ;
} // Trap*/
```

改得更简洁

```
global_result = 0.0;
...
#pragma omp parallel num_threads(thread_count)
  Trap(a, b, n, &global_result);
...

```

有什么问题？

```
global_result = 0.0;
#pragma omp parallel num_threads(thread_count)
{
  #pragma omp critical
  global_result += local_trap(a, b, n);
}

```

不同线程的全部计算被串行化了

避免所有计算串行化

```
global_result = 0.0;
#pragma omp parallel num_threads(thread_count)
{
    # pragma omp critical
    global_result += Local_trap(a, b, n);
}

global_result = 0.0;
#pragma omp parallel num_threads(thread_count)
{
    double my_result = 0.0; /* private */
    my_result += Local_trap(a, b, n);
    # pragma omp critical
    global_result += my_result;
}
```

不同线程并行计算
只有全局求和串行

https://blog.csdn.net/welzin_4099651

OpenMP 归约

改为使用归约

```
global_result = 0.0;
#pragma omp parallel num_threads(thread_count)
{
    double my_result = 0.0; /* private */
    my_result += Local_trap(a, b, n);
    # pragma omp critical
    global_result += my_result;
}

global_result = 0.0;
#pragma omp parallel num_threads(thread_count)
    reduction(+: global_result)
    global_result += Local_trap(a, b, n);
```

私有变量等繁琐工作编译器代劳
全局求和采用递归算法

- ****对for循环自动进行并行化. 默认块状划分****

OpenMP支持归约操作

- 归约就是将相同的规约操作符重复地应用到操作数序列来得到一个结果的计算。

```
sum = 0;
#pragma omp parallel for reduction(+:sum)
for (i=0; i < 100; i++) {
    sum += array[i];
}
```

支持的归约运算及初值:

+	0	位与 & ~0	逻辑与 & 1
-	0	位或 0	逻辑或 0
*	1	位异或 ^ 0	

Serial Program:	Parallel Program:
<pre> void main() { double Res[1000]; for(int i=0;i<1000;i++) { do_huge_comp(Res[i]); } } </pre>	<pre> void main() { double Res[1000]; #pragma omp parallel for for(int i=0;i<1000;i++) { do_huge_comp(Res[i]); } } </pre>

局限和语义

for	{		index++
			++index
			index--
			--index
		index = start ;	index += incr
			index -= incr
			index = index + incr
			index = incr + index
		index = index - incr	
	}		

○并非所有“元素级”循环都能并行化

```
#pragma omp parallel for
for (i=0; i < numPixels; i++) {}
```

- 循环变量：带符号整数
- 终止检测：<, <=, >, >=与循环不变量
- 每步迭代递增/递减一个循环不变量
- 对<, <=向上计数；对>, >=向下计数
- 循环体：无进/出控制流

- 第二个错了,不能编译通过
- 第一个错了,可以编译通过,但是结果会错误.

- 创建线程，在线程间分配循环步；要求迭代次数可预测。

- ❑ 不检查依赖性!
- ❑ 不支持while、do-while、循环体包含break等

简单并行化循环的版本

```
/* Input: a, b, n */
h = (b - a)/n;
approx = (f(a) + f(b))/2.0;
for (i = 1; i <= n-1; i++)
    approx += f(a + i*h);
approx = h * approx;
```



```
/* Input: a, b, n */
h = (b - a)/n;
approx = (f(a) + f(b))/2.0;
#pragma omp parallel for num_threads(thread_count) \
    reduction(+: approx)
for (i = 1; i <= n-1; i++)
    approx += f(a + i*h);
approx = h * approx;
```

- 除非最开始出现, 后面紧接for循环

```
1 | #pragma omp parallel for
2 | 否则可以不要parallel
```

○ 语法

#pragma omp for [clause [clause] ...]
for 循环

○ 子句形式

shared(list)
private(list)
reduction(operator:list)
schedule(type[, chunk])
nowait (C/C++: 用于#pragma omp for)

#pragma omp parallel private(f) {
f=7;
#pragma omp for
for (i=0; i<20; i++)
a[i] = b[i] + f * (i+1);
} /* omp end parallel */

https://blog.csdn.net/weixin_40996518

一些定义, 定理

- 计算等价
 - 相同的输入, 产生相同的输出(顺序也同)
- 重排转换
 - 改变语句的执行顺序, 不增加或删除任何语句的执行
- 一个重排转换保持依赖关系
 - 重排转换过程中保持了依赖源和目的语句的相对执行顺序

依赖关系基本定理

- 任何重排转换, 只要保持了程序中所有依赖关系, 它就保持了程序的含义

数组的数据依赖

```
for (i=2; i<5; i++)  
A[i] = A[i-2]+1;
```

循环进位
依赖关系

```
for (i=1; i<=3; i++)  
A[i] = i+1;  
B[i] = A[i]*3
```

循环独立
关系

识别并行循环（直观地）

- 寻找循环中的数据依赖
- 没有依赖关系跨越迭代步边界→并行化是安全的

估算π

$$\pi = 4 \left[1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots \right] = 4 \sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}$$

```
double factor = 1.0;  
double sum = 0.0;  
for(k=0; k<n; k++){  
sum += factor/(2*k+1);  
factor = -factor;  
}  
pi_approx = 4.0*sum;
```

https://blog.csdn.net/weixin_40996518

尝试第一个OpenMP版本

```
double factor = 1.0;
double sum = 0.0;
# pragma omp parallel for num_threads(thread_count) \
    reduction(+:sum)
for (k = 0; k < n; k++) {
    sum += factor/(2*k+1);
    factor = -factor;
}
pi_approx = 4.0*sum;
```

有什么问题？

一条语句由多个线程
并行执行产生数据依赖

https://blog.csdn.net/weixin_40996518

根据k的奇偶性计算factor

○ k为奇数，factor为-1；k为偶数，factor为+1

```
double factor = 1.0;
double sum = 0.0;
# pragma omp parallel for num_threads(thread_count) \
    reduction(+:sum)
for (k = 0; k < n; k++) {
    factor = (k % 2 == 0) ? 1.0 : -1.0;
    sum += factor/(2*k+1);
}
pi_approx = 4.0*sum;
```

仍是错误的！

factor应是私有的！

https://blog.csdn.net/weixin_40996518

factor变为私有

```
double sum = 0.0;
# pragma omp parallel for num_threads(thread_count) \
    reduction(+:sum) private(factor)
for (k = 0; k < n; k++) {
    factor = (k % 2 == 0) ? 1.0 : -1.0;
    sum += factor/(2*k+1);
}
pi_approx = 4.0*sum;
```

https://blog.csdn.net/weixin_40996518

- 将# pragma omp parallel 提到最外层循环
- **pragma omp for** 指针对没有循环依赖的内部两个循环

避免线程创建、销毁开销

```
# pragma omp parallel num_threads(thread_count) \
    default(none) shared(a, n) private(i, tmp, phase)
for (phase = 0; phase < n; phase++) {
    if (phase % 2 == 0)
    # pragma omp for
        for (i = 1; i < n; i += 2) {
            ...
        }
    else
    # pragma omp for
        for (i = 1; i < n - 1; i += 2) {
            ...
        }
}
```

https://blog.csdn.net/weixin_40996518

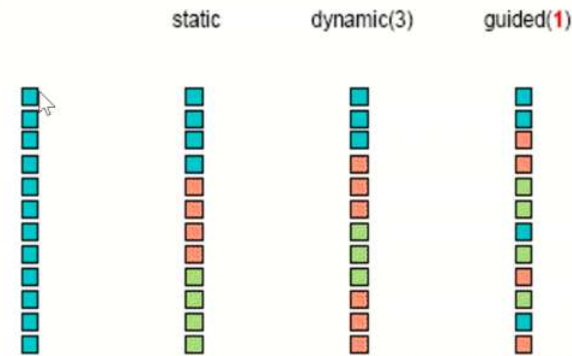
循环调度

编程模型——循环调度

○ schedule子句确定如何在线程间划分循环

- static([chunk]) 静态划分
 - 分配给每个线程 [chunk]步迭代，所有线程都分配完后继续循环分配，直至所有迭代步分配完毕
 - 默认[chunk]为ceil(#iterations/#threads)
- dynmaic([chunk]) 动态划分
 - 分给每个线程[chunk]步迭代，一个线程完成任务后再为其分配 [chunk]步迭代
 - 逻辑上形成一个任务池，包含所有迭代步
 - 默认[chunk]为1
- guided([chunk]) 动态划分，但划分过程中[chunk]指数减小
 - 类似于DYNAMIC调度，但分块开始大，随着迭代分块越来越少，循环区间的划分是基于类似下列公式完成的（不同的编译系统可能不同）：
$$S_k = \left\lceil \frac{R_k}{2^N} \right\rceil$$
其中N是线程个数，Sk表示第k块的大小，Rk是剩余下未被调度的循环迭代次数。

循环调度



调度子句

○ 默认调度

```
sum = 0.0;
# pragma omp parallel for num_threads(thread_count) reduction(+:sum)
for (i = 0; i <= n; i++)
    sum += f(i);
```

等价于

```
sum = 0.0;
# pragma omp parallel for num_threads(thread_count) reduction(+:sum)
    schedule(static, n/thread_count)
for (i = 0; i <= n; i++)
    sum += f(i);
```

MPI编程

- 一个库
- SPMD
- 隔离了独立的地址空间
 - 不会有数据竞争, 但可能有通信错误
 - 编程复杂, 代码膨胀
 - 暴露了执行模型, 迫使程序员思考局部性, 亮点对性能都有好处
- 所有通信, 同步都需要调用函数完成
 - 无共享变量

基本原语

- 1 MPI_Init(&argc, &argv); 初始化MPI
- 2 MPI_Finalize(); 结束MPI
- 3 MPI_Comm_size(MPI_COMM_WORLD, &myid)
- 4 查看进程数

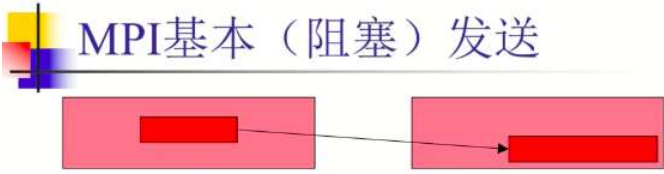

```

5 MPI_Comm_rank(MPI_COMM_WORLD, &numprocs)
6 查看进程号
7
8 第二个参数是赋值操作

```

通信

MPI基本（阻塞）发送



MPI_Send(A, 10, MPI_DOUBLE, 1, ...) MPI_Recv(B, 20, MPI_DOUBLE, 0, ...)

```

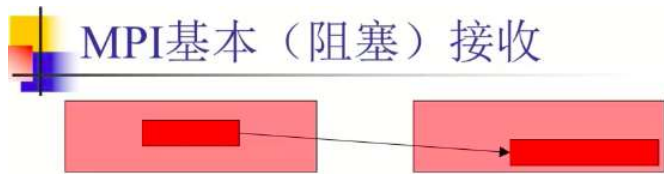
int MPI_Send(void* buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)

```

- 消息缓冲区(buf, count, datatype)
- 目的进程dest——目的进程在comm指定的通信域中的编号
- 阻塞发送——函数返回时，数据已经转给系统进行发送，缓冲区可作他用；消息可能还未送达目的进程

https://blog.csdn.net/weixin_40996518

MPI基本（阻塞）接收



MPI_Send(A, 10, MPI_DOUBLE, 1, ...) MPI_Recv(B, 20, MPI_DOUBLE, 0, ...)

```

int MPI_Recv(void* buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm, MPI_Status *status)

```

- 阻塞接收——等待，直至收到匹配的（source和tag都相同）的消息，缓冲可作他用
- source可以是comm中的编号，或MPI_ANY_SOURCE
- tag为特定标签（需匹配）或MPI_ANY_TAG
- 接收的数据量比指定的少是允许的，但接收到更多数据就是错误
- status包含更多信息（如接收到的消息大小）

https://blog.csdn.net/weixin_40996518

一些基本概念

- 如何组织进程
 - 进程组成进程组
 - 每个消息都是在一个特定上下文中发送，必须在同一个上下文中接收
 - 进程组和上下文一起形成了通信域(communicator)
 - 进程用它在进程组（与某个通信域关联）中的编号标识
- MPI_COMM_WORLD: 默认通信域，其进程组包含所有初始进程

https://blog.csdn.net/weixin_40996518

- 消息中的数据用三元组描述：地址, 个数, 类型


```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
    int rank, size, i, buf[1];
    MPI_Status status;
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &rank );
    MPI_Comm_size( MPI_COMM_WORLD, &size );

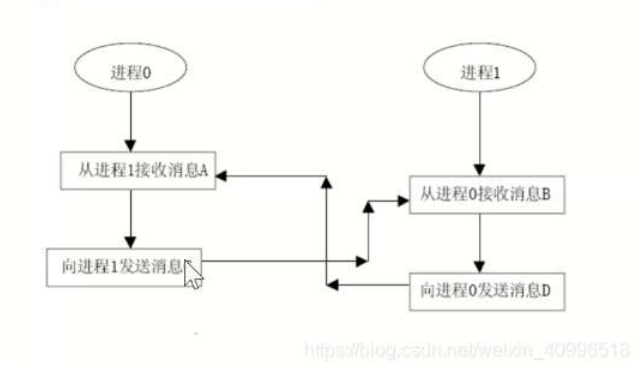
    if (rank == 0) {
        for (i=0; i<(size-1); i++) {
            MPI_Recv( buf, 1, MPI_INT, MPI_ANY_SOURCE,
                      MPI_ANY_TAG, MPI_COMM_WORLD, &status );
            printf( "Msg=%d from %d with tag %d\n",
                    buf[0], status.MPI_SOURCE, status.MPI_TAG );
        }
    }
    else {
        buf[0]=rank;
        MPI_Send( buf, 1, MPI_INT, 0, 100-rank, MPI_COMM_WORLD );
    }
    MPI_Finalize();
    return 0;
}
```

阻塞通信模式

- MPI有一个默认缓冲区, 执行Send时, 先将数据放入缓冲区, 只要缓冲区中的数据被发送完毕, 就认为发送完毕, 可以继续执行其余代码.
- 接收方一定是等到数据全部接受完毕才执行其余代码
- 缺点
 - 死锁

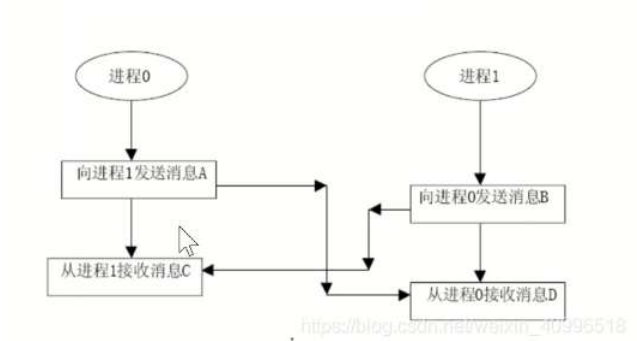
通信模式——避免死锁

必然死锁的通信模式



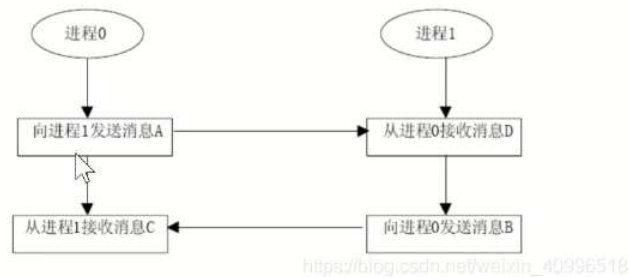
不安全的通信模式

缓冲区问题可能导致死锁





安全的通信模式



两种编程模型

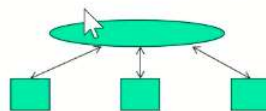
- 对等式
- 主从式

消息传递两种编程模型

- 对等式（地位平等，功能相近）



- 主从式（地位不同，功能不同）



https://blog.csdn.net/weixin_40996518

组通信

- 一个进程组内所有进程同时参加通信
- 所有参与进程的函数调用形式一致
- 三个功能：通信，同步，计算
- 在组通信中不需要消息标志参数
- 操作是成对的：互为逆操作

广播和归约

n-1 1-n



广播和归约

○ one-to-all broadcast

- 一个进程向其他所有进程发送相同数据
- 初始，只有源进程有一份m个字的数据
- 广播操作后，所有进程都有一份相同数据

○ 对应操作：all-to-one reduction

- 初始，每个进程都有一份m个字的数据
- 归约操作后，p份数据经过计算（加、乘、...）得到一份数据（结果），传送到目的进程

- 应用：矩阵相乘、高斯消去、最短路径、向量内积...

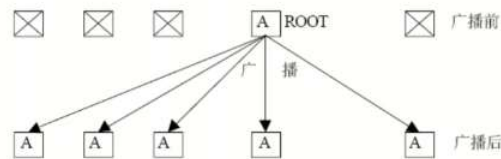
https://blog.csdn.net/weixin_40996518

20

MPI广播

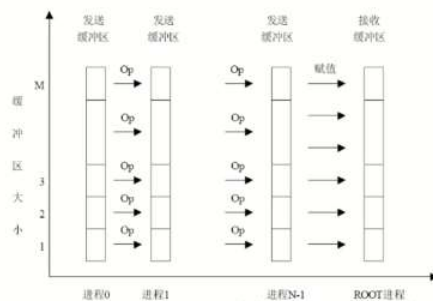
```
int MPI_Bcast(void* buffer,int count,MPI_Datatype
              datatype,int root, MPI_Comm comm)
```

- root: 每个调用广播操作的进程中root都相同
- count: 待广播的数据的个数



MPI归约

```
int MPI_Reduce(void* sendbuf, void* recvbuf, int count,
MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm
comm)
```



https://blog.csdn.net/welxin_40996518

MPI预定义归约操作

名字	含义
MPI_MAX	最大值
MPI_MIN	最小值
MPI_SUM	求和
MPI_PROD	求积
MPI_LAND	逻辑与
MPI_BAND	按位与
MPI_LOR	逻辑或
MPI_BOR	按位或
MPI_LXOR	逻辑异或
MPI_BXOR	按位异或
MPI_MAXLOC	最大值且相应位置
MPI_MINLOC	最小值且相应位置

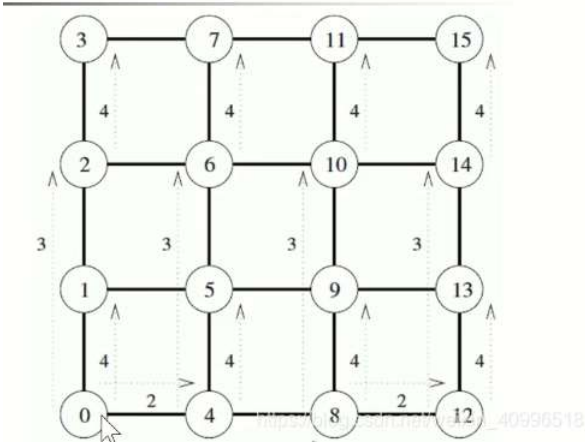
操作	允许的数据类型
MPI_MAX, MPI_MIN	C整数, Fortran整数, 浮点数
MPI_SUM, MPI_PROD	C整数, Fortran整数, 浮点数, 复数
MPI_LAND, MPI_LOR, MPI_LXOR	C整数, 逻辑型
MPI_BAND, MPI BOR, MPI BXOR	C整数, Fortran整数, 字节型

18

环/线性阵列

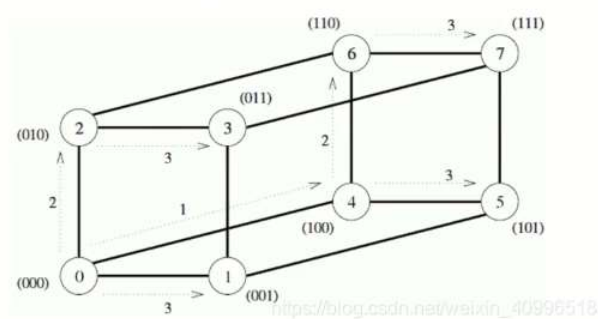
- 简单算法
 - 源进程顺序将数据发送给其它 $p-1$ 个进程
 - 低效，源进程成为瓶颈，网络利用不充分
- 递归加倍法
 - 第一步：源进程 $P \rightarrow$ 进程 P'
 - 第二步：进程 $P, P' \rightarrow$ 另外两个进程...
- 避免冲突，环的实现
 - 第一步：源进程 $P \rightarrow$ 距离最远 ($p/2$) 的进程
 - 第二步：两个进程 $\rightarrow$ 距离 $p/4$ 的进程...

主/距离 $p/4$ 的近似...



超立方广播算法

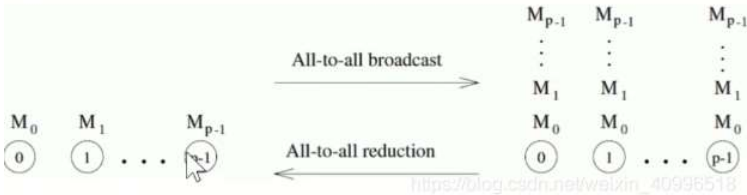
- d维mesh, 每个维度2个节点
- 分为d个阶段, 每个阶段1个步骤



n-n

All-to-All广播和归约

- All-to-All广播: 所有进程同时发起一个广播, 每个进程向所有其他进程发送m个字
- All-to-All归约
- 应用: 矩阵相乘, 矩阵向量相乘



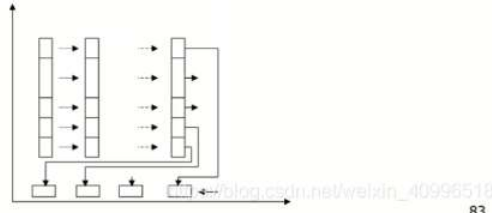
组收集

- int MPI_Allgather(void* sendbuf, int sendcount, MPI_Datatype sendtype,void* recvbuf, int recvcount, MPI_Datatype recvtype,MPI_Comm comm)
- MPI_Allgatherv
- 相当于组内每个进程都执行一次收集

归约并散发

```
int MPI_Reduce_scatter(void* sendbuf, void* recvbuf,
                      int *recvcounts MPI_Datatype datatype,
                      MPI_Op op, MPI_Comm comm)
```

- 归约后再执行一次散发操作

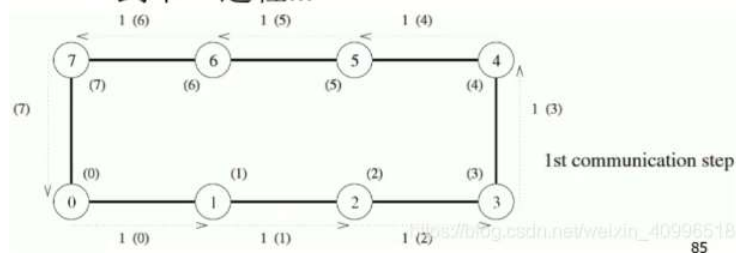


算法思想

- 简单方法: p 个节点各执行一个one-to-all广播, 性能差, 可能时间是一个one-to-all广播的 p 倍
- 更有效地利用链路: p 个all-to-all广播同时进行, 一个链路上同时传输的消息合并为大消息

线性阵列和环

- 保证每个节点总有数据传输给邻居——链路总保持忙
- 第一步：每个进程→某个邻居进程
- 第二步：每个进程将刚收到的数据转发到下一进程...



环all-to-all归约

- 初始，每个进程都有 p 个消息（ m 个字）
- 每个消息会与其它 $p-1$ 个进程的各1个消息进行计算，最终保存在某个进程
- 实现：all-to-all广播实现的逆过程

- all归约, 最后每个节点结果相同
- all-to-all归约, 每个节点结果不同

A11归约与前缀和

- all-to-one归约+one-to-all广播——完成后，每个进程都有相同的归约结果
- 与all-to-all归约不同——p个all-to-one归约，每个进程得到不同归约的结果
- 应用：同步操作实现
- 快速算法：利用all-to-all广播方法，每个步骤不是将消息组合，而是进行运算

$$T = (t_s + t_w m) \log p$$

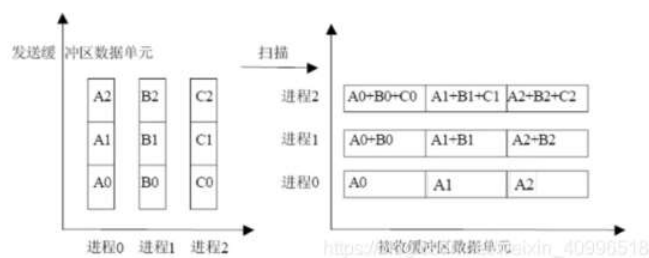
前綴和

- 操作之前，进程 P_k 保存数 n_k ，完成后前缀和操作后，保存 $\sum_{i=0}^k n_i$
- 利用 all-to-all 广播方法
 - 每个步骤进行加法，而不是组合消息
 - 两个缓冲
 - result——本节点最终结果，只累加编号小于自己的节点发送来的数据
 - 传输缓冲：累加所有收到的数据

扫描

```
int MPI_Scan(void* sendbuf, void* recvbuf, int count,
             MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
```

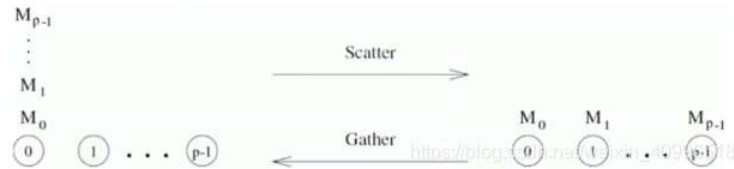
- 可以将扫描看作是一种特殊的归约，即每一个进程都对排在它前面的进程进行归约操作



Scatter 和 Gather

Scatter和Gather

- **scatter**: 源节点向其它每个节点发送不同数据, one-to-all 个体化通信
与 one-to-all 广播 (发送相同数据) 不同
- 对应操作: **gather, concatenation**
目的节点从其他每个节点接收不同数据
不同于 all-to-one 归约, 无合并/归约运算



非阻塞通信

- 发送和接受数据的过程中, CPU可以执行别的事情.
- 查询命令MPI_Wait 查看是否完成

消息传递与缓冲

- 当用户提供的buffer可被重用时，才表明send完成

```
*buf = 3;
MPI_Send(buf, 1, MPI_INT ...)
*buf = 4; /* OK, 接收方总是会收到3 */
```

```
*buf = 3;
MPI_Isend(buf, 1, MPI_INT ...)
*buf = 4; /* 不确定接收方收到3还是4 */
MPI_Wait(...);
```

- send完成并不意味着receive也完成
 - 消息可能被系统缓冲
 - 消息可能还在传输中

单一非阻塞通信的完成

- `int MPI_Wait(MPI_Request *request, MPI_Status *status)`
 - 等待完成并释放该对象，返回信息放在 `status` 中
 - 若为发送操作，则发送完成，发送缓冲区可以重用
 - 若为接收操作，则数据已经接收到，放在接收缓冲区中
- 若通信尚未完成，相当于回到阻塞状态

相当于回到阻塞状态

多个非阻塞通信的完成

- `int MPI_Waitany(int count, MPI_Request *array_of_requests, int *index, MPI_Status *status)`
 - `count`: 非阻塞通信对象的个数
 - `array_of_requests`: 非阻塞通信完成对象数组
 - `index`: 完成对象对应的句柄索引
 - `status`: 返回状态
- 任何一个对象完成就返回

https://blog.csdn.net/weixin_40996518

多个非阻塞通信的完成(2)

- `int MPI_Waitall(int count, MPI_Request *array_of_requests, MPI_Status *array_of_statuses)`
 - `count`: 非阻塞通信对象的个数
 - `array_of_requests`: 非阻塞通信完成对象数组
 - `array_of_statuses`: 状态数组
- 所有对象均完成才返回

多个非阻塞通信的完成(3)

- `int MPI_Waitsome(int incount, MPI_Request *array_of_request, int *outcount, int *array_of_indices, MPI_Status *array_of_statuses)`
 - `incount`: 非阻塞通信对象的个数
 - `array_of_request`: 非阻塞通信对象数组
 - `outcount`: 已完成对象的数目
 - `array_of_indices`: 已完成对象的下标数组
 - `array_of_statuses`: 已完成对象的状态数组
- 只要有不为0的对象完成就返回

- 只是一个查询, 不满足也不会变阻塞.

- 并不等到通信结束，立即返回状态
- 单个
 - `int MPI_Test(MPI_Request*request, int *flag, MPI_Status *status)`
 - `flag=true`，表示指定通信操作已完成
 - `flag=false`，表示指定通信操作还未结束
- 多个
 - 与Wait相对应，只是增加了flag
 - `MPI_Testsome`并无flag参数

Lnag参数

非阻塞通信对象

- 识别各种通信操作，判断相应的非阻塞操作是否完成
- 释放
 - 确认一个非阻塞通信操作完成时，可直接释放该对象所占用的资源，而不是通过调用非阻塞通信完成操作来间接地释放
 - `int MPI_Request_free(MPI_Request * request)`
 - 如果与该非阻塞通信对象相联系的通信还没有完成则该对象的资源并不会立即释放

https://blog.csdn.net/weixin_40996518

非阻塞通信的取消

- `int MPI_Cancel(MPI_Request *request)`
 - 若取消操作调用时相应的非阻塞通信已经开始则它会正常完成
 - 也必须调用非阻塞通信的完成操作或查询对象的释放操作来释放查询对象
- `int MPI_Test_cancelled(MPI_Status status, int *flag)`
 - 如果 `flag=true` 则表明该通信已经被成功取消

https://blog.csdn.net/weixin_40996518

混合编程

- MPI描述了进程（独立地址空间）间并行
- 线程并行则是提供了一个进程内的共享内存模型
- 多核集群的常见编程方法
 - 纯MPI
 - 节点内和跨节点都采用MPI实现进程并行
 - 节点内MPI内部都是采用共享内存来通信的
 - MPI + OpenMP
 - 节点内使用OpenMP，跨节点使用MPI
 - MPI + Pthreads
 - 节点内使用Pthreads，跨节点使用MPI
- 后两种方法被称为“混合编程”

https://blog.csdn.net/weixin_40996518

MPI四种线程安全级别

- MPI定义了四级线程安全级别——这其实是应用程序向MPI做出的承诺
 - `MPI_THREAD_SINGLE`：应用中只有一个线程
 - `MPI_THREAD_FUNNELED`：多线程，但只有主线程会进行MPI调用（调用 `MPI_Init_thread` 的那个线程）
 - `MPI_THREAD_SERIALIZED`：多线程，但同时只有一个线程会进行MPI调用
 - `MPI_THREAD_MULTIPLE`：多线程，且任何线程任何时候都会进行MPI调用（有一些限制避免竞争条件）
- 线程安全级别是递增顺序的
 - 如果一个应用工作在FUNNELED模式，它也工作在SERIALIZED模式
- MPI定义了一个 `MPI_Init` 的替代API
 - `MPI_Init_thread(requested, provided)`
 - 应用给出它希望的级别；MPI实现返回它支持的级别

https://blog.csdn.net/weixin_40996518

线程和MPI

- 并不要求一个MPI实现支持高于MPI_THREAD_SINGLE的级别；即，不要求实现是线程安全的
- 一个完全线程安全的实现会支持MPI_THREAD_MULTIPLE
- 一个调用MPI_Init（而非MPI_Init_thread）的程序应假定实现只支持MPI_THREAD_SINGLE
- 一个未调用MPI_Init_thread的多线程MPI程序是一个错误程序（很常见的错误）

MPI_THREAD_MULTIPLE规范

- **序：**当多个线程并发调用MPI API时，结果就像以某种（任意）顺序串行执行调用一样
 - 序是在每个线程内维持的
 - 程序员必须确保在相同通信域、窗口或文件上的组操作在线程间正确排序
 - 例如，不能在相同通信域上由一个线程调用广播操作、而同时另一个线程调用归约操作
 - 当一个应用内的线程进行冲突的MPI调用时，程序员应负责防止竞争条件
 - 例如，一个线程访问一个info对象、而同时另一个线程在释放它
- **阻塞：**阻塞MPI调用只会阻塞调用线程，而不会阻止其他线程的运行、也不会阻止它们调用MPI函数